

Data Normalization

Data normalization is the practice of organizing data entries to ensure they appear similar across all fields and records, making information easier to find, group and analyze. There are many data normalization techniques and rules.

Normalizing the data refers to scaling the data values to a much smaller range such as [-1, 1] or [0.0, 1.0]. There are different methods to normalize the data, as discussed below.

Normalization is an essential step in the preprocessing of data for machine learning models, and it is a feature scaling technique.

Normalization is especially crucial for data manipulation, scaling down, or up the range of data before it is utilized for subsequent stages in the fields of soft computing, cloud computing, etc.

Min-max scaling and Z-Score Normalisation (Standardisation) are the two methods most frequently used for normalization in feature scaling.

***Normalization** is scaling the data to be analyzed to a specific range such as [0.0, 1.0] to provide better results.*

Data Normalization Techniques

1- Min-Max normalization:

This method of normalising data involves transforming the original data linearly.

The data's minimum and maximum values are obtained, and each value is then changed using the formula that follows.

$$X_{\text{normalization}} = \frac{X - x_{\min}}{x_{\max} - x_{\min}}$$

This division scales the variable to a proportion of the entire range.

As a result, the normalized value falls between 0 and 1.

This normalization guarantees all features will have exact same scale but does not handle outliers well.

2- Normalisation of Z-score or Zero Mean (Standardisation):

Using the mean and standard deviation of the data, values are normalised in this technique to create a standard normal distribution (mean: 0, standard deviation: 1). The equation that is applied is:

$$Z = \frac{x - \mu}{\sigma}$$

z-score handles outliers, but does not produce normalized data with the exact same scale.

Example:

Normalize the following group of data with min-max normalization 1000, 2000, 3000, 9000

Solution:

$$x_{\min} = 1000, x_{\max} = 9000$$

No.	x	X _{norm}
1	1000	$(1000-1000)/(9000-1000)=0$
2	2000	$(2000-1000)/(9000-1000)=0.125$
3	3000	$(3000-1000)/(9000-1000)=0.25$
4	9000	$(9000-1000)/(9000-1000)=1$

Example:

Perform z-score normalization with the data set: 3, 5, 5, 8, 9, 12, 12, 13, 15, 16, 17, 19, 22, 24, 25, 134

Solution:

Mean $\mu = \text{summation}/n = 21.2$, standard deviation $\sigma = \sqrt{\frac{\sum(x-\mu)^2}{n}} = 29.8$

No.	X	X _{norm}
1	3	$(3-21.2)/29.8=-0.61$

2	5	$(5-21.2)/29.8=-0.54$
3	5	$(5-21.2)/29.8=-0.54$
4	8	$(8-21.2)/29.8=-0.44$
5	9	$(9-21.2)/29.8=-0.41$
6	12	$(12-21.2)/29.8=-0.31$
7	12	$(12-21.2)/29.8=-0.31$
8	13	$(13-21.2)/29.8=-0.28$
9	15	$(15-21.2)/29.8=-0.21$
10	16	$(16-21.2)/29.8=-0.17$
11	17	$(17-21.2)/29.8=-0.14$
12	19	$(19-21.2)/29.8=-0.07$
13	22	$(22-21.2)/29.8=0.03$
14	24	$(24-21.2)/29.8=0.09$
15	25	$(25-21.2)/29.8=0.13$
16	134	$(134-21.2)/29.8=3.79$

Pandas Package

The **pandas** package is the most important tool at the disposal of Data Scientists and Analysts working in Python today. **pandas** is the backbone of most data projects.

Through **pandas**, you get acquainted with your data by **cleaning**, **transforming**, and **analyzing** it.

For example, say you want to explore a dataset stored in a CSV on your computer. Pandas will extract the data from that CSV into a **DataFrame** — a table, basically — then let you do things like:

- **Calculate statistics** and answer questions about the data, like
 - What's the average, median, max, or min of each column?
 - Does column A correlate with column B?
 - What does the distribution of data in column C look like?
- **Clean** the data by doing things like removing missing values and filtering rows or columns by some criteria

- **Visualize** the data with help from **Matplotlib**. Plot bars, lines, histograms, bubbles, and more.
- Store the cleaned, transformed data back into a CSV, other file or database.

Pandas is built on top of the **NumPy** package, meaning a lot of the structure of NumPy is used or replicated in Pandas.

Data in pandas is often used to feed statistical analysis in **SciPy**, plotting functions from **Matplotlib**, and machine learning algorithms in **Scikit-learn**.

To import **pandas** we usually import it with a shorter name since it's used so much:

```
import pandas as pd
```

The primary two components of pandas are the **Series** and **DataFrame**.

A **Series** is essentially a **column**, and a **DataFrame** is a **multi-dimensional table** made up of a collection of Series.

Series			Series			DataFrame	
apples			oranges			apples oranges	
0	3	+	0	0	=	0	3 0
1	2		1	3		1	2 3
2	0		2	7		2	0 7
3	1		3	2		3	1 2

There are *many* ways to create a **DataFrame** with pandas, but a great option is to just use a simple **dict**.

Let's say we have a fruit stand that sells apples and oranges. We want to have a column for each fruit and a row for each customer purchase. To organize this as a dictionary for pandas:

```
data = {
    'apples': [3, 2, 0, 1],
    'oranges': [0, 3, 7, 2]
}
```

and then pass it to the pandas DataFrame constructor:

```
purchases = pd.DataFrame(data)
```

```
print(purchases)
```

```
In [5]: data = {
        'apples': [3, 2, 0, 1],
        'oranges': [0, 3, 7, 2]
        }

        purchases = pd.DataFrame(data)

        print(purchases)
```

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Note: index of this dataframe is [0-3]

but we could also create our own when we initialize the DataFrame.

Let's have customer **names** as our index:

```
purchases = pd.DataFrame(data, index=['June', 'Robert', 'Lily', 'David'])
```

```
purchases
```

```
In [6]: purchases = pd.DataFrame(data, index=['June', 'Robert', 'Lily', 'David'])
        purchases
```

Out[6]:

	apples	oranges
June	3	0
Robert	2	3
Lily	0	7
David	1	2

So now we could **locate** a customer's order by using their name:

```
purchases.loc['June']
```

```
In [7]: purchases.loc['June']  
  
Out[7]: apples      3  
        oranges     0  
        Name: June, dtype: int64
```

```
In [ ]: |
```

Writing data into CSVs

df.to_csv('purchases.csv')

Reading data from CSVs

With CSV files all you need is a single line to load in the data:

```
df = pd.read_csv('purchases.csv')
```

df

```
In [9]: df.to_csv('purchases.csv')  
  
In [10]: df = pd.read_csv('purchases.csv')  
         df  
  
Out[10]:
```

	Unnamed: 0	Name	Age
0	0	Amit	20
1	1	Cody	21
2	2	Drew	25

```
In [ ]:
```

create data frame with pandas

import pandas as pd

```
d = {'col1': [1, 2, 3, 4, 7], 'col2': [4, 5, 6, 9, 5], 'col3': [7, 8, 12, 1, 11]}
```

```
df = pd.DataFrame(data=d)
```

```
print(df)
```

Creating DataFrame to Export Pandas DataFrame to CSV

```
# importing pandas as pd
```

```
import pandas as pd
```

```
# list of name, degree, score
```

```
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
```

```
deg = ["MBA", "BCA", "M.Tech", "MBA"]
```

```
scr = [90, 40, 80, 98]
```

```
# dictionary of lists
```

```
dict = {'name': nme, 'degree': deg, 'score': scr}
```

```
df = pd.DataFrame(dict)
```

```
print(df)
```

```
# saving the dataframe
```

	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

```
df.to_csv('file1.csv')
```

```
In [2]: # importing pandas as pd
import pandas as pd

# List of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)

print(df)
```

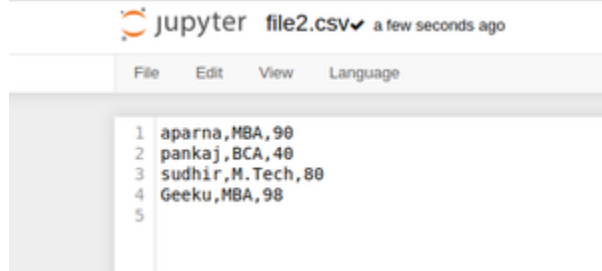
	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

```
In [ ]:
```

Saving CSV without *headers* and *index*.

saving the dataframe

```
df.to_csv('file2.csv', header=False, index=False)
```



Save the CSV file to a specified location

saving the dataframe

```
df.to_csv(r'C:\Users\Admin\Desktop\file3.csv')
```

Write a DataFrame to CSV file using tab separator

```
import pandas as pd
import numpy as np
```

```
users = {'Name': ['Amit', 'Cody', 'Drew'],
        'Age': [20, 21, 25]}
```

#create DataFrame

```
df = pd.DataFrame(users, columns=['Name', 'Age'])
```

```
print("Original DataFrame:")
print(df)
print('Data from Users.csv:')
```

```
df.to_csv('Users.csv', sep='\t', index=False, header=True)
new_df = pd.read_csv('Users.csv')
```

```
print(new_df)
```



```
In [4]: import pandas as pd
import numpy as np

users = {'Name': ['Amit', 'Cody', 'Drew'],
        'Age': [20, 21, 25]}

#create DataFrame
df = pd.DataFrame(users, columns=['Name', 'Age'])

print("Original DataFrame:")
print(df)
print('Data from Users.csv:')

df.to_csv('Users.csv', sep='\t', index=False, header=True)
new_df = pd.read_csv('Users.csv')

print(new_df)
```

Original DataFrame:

	Name	Age
0	Amit	20
1	Cody	21
2	Drew	25

Data from Users.csv:

	Name\tAge
0	Amit\t20
1	Cody\t21
2	Drew\t25

Or

Save dataframe to json file:

df.to_json('purchases.json')

To read it:

df = pd.read_json('purchases.json')

df

Reading data from a SQL database

If you're working with data from a SQL database you need to first establish a **connection** using an appropriate Python library, then [pass a query to pandas](#). Here we'll use **SQLite** to demonstrate.

```
import sqlite3
```

```
con = sqlite3.connect("database.db")
```

```
df.to_sql('purchases', con) # now the dataframe df is stored in a database file
```

In this SQLite database we have a table called *purchases*, and our index is in a column called "index".

By passing a **SELECT** query and our con, we can read from the *purchases* table:

```
import sqlite3
```

```
con = sqlite3.connect("database.db")
```

```
df = pd.read_sql_query("SELECT * FROM purchases", con)
```

```
df # to print
```

How To Convert Sklearn Dataset To Pandas Dataframe in Python?

```
from sklearn.datasets import load_iris
```

```
import pandas as pd
```

```
# Load the iris dataset from sklearn using load_iris() method
```

```
iris = load_iris()
```

```
# Convert the iris dataset to a pandas dataframe
```

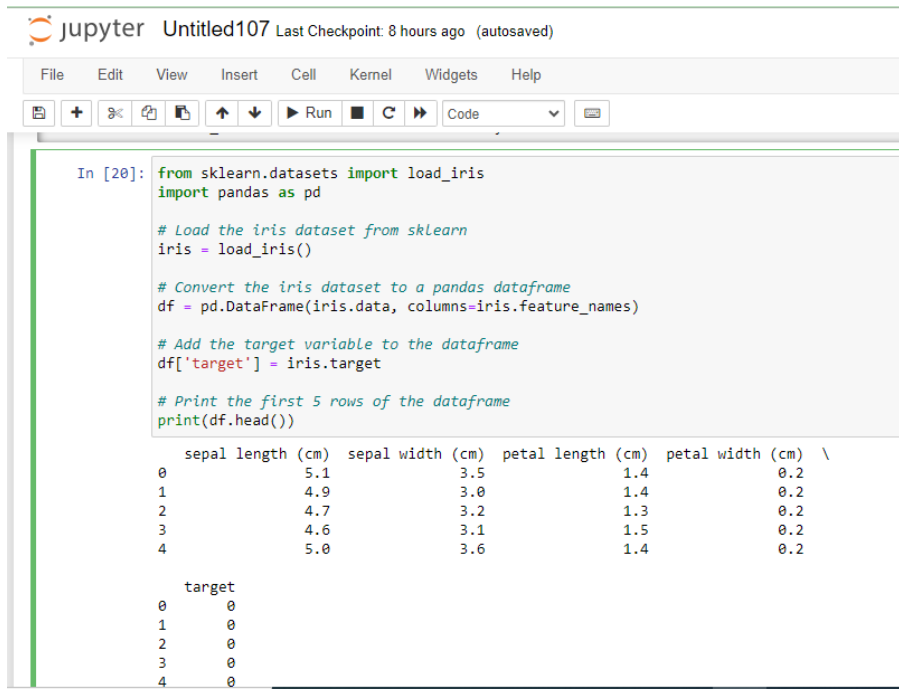
```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
```

Add the target variable to the dataframe

```
df['target'] = iris.target
```

Print the first 5 rows of the dataframe

```
print(df.head())
```



The screenshot shows a Jupyter Notebook titled 'Untitled107' with a toolbar at the top containing icons for File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. Below the toolbar is a code editor with the following Python code:

```
In [20]: from sklearn.datasets import load_iris
import pandas as pd

# Load the iris dataset from sklearn
iris = load_iris()

# Convert the iris dataset to a pandas dataframe
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add the target variable to the dataframe
df['target'] = iris.target

# Print the first 5 rows of the dataframe
print(df.head())
```

The output of the code is displayed below the code editor, showing the first 5 rows of the Iris dataset as a pandas DataFrame:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

Below the table, the 'target' column is shown for the same 5 rows:

	target
0	0
1	0
2	0
3	0
4	0

Explore the dataset:

```
print(df.info())
```

```
print(df.describe())
```

The **info()** function is useful to understand the overall structure of the data frame, the number of non-null values in each column, and the memory usage. While the summary statistics provide an overview of numerical features in your dataset.

```
[42]: print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column              Non-Null Count  Dtype  
---  --
0   sepal length (cm)    150 non-null    float64
1   sepal width (cm)     150 non-null    float64
2   petal length (cm)    150 non-null    float64
3   petal width (cm)     150 non-null    float64
4   target               150 non-null    int32   
dtypes: float64(4), int32(1)
memory usage: 5.4 KB
None
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

	petal width (cm)	target
count	150.000000	150.000000
mean	1.199333	1.000000
std	0.762238	0.819232
min	0.100000	0.000000
25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

In []:

Removing Missing Values (if dataset has missing)

`df.dropna(inplace=True)`

or

by replacing:

`df = df.replace(1, 31) # Replace 1 with 31`

`df = df.replace(1, np.nan) # Replace 1 with "np.nan"`

Removing Duplicates

`duplicate_rows = df.duplicated()`

```
print("Number of duplicate rows:", duplicate_rows.sum())
```

```
# Number of duplicate rows: 1
```

The duplicates can be removed via the **drop_duplicates()** function.

```
df.drop_duplicates(inplace=True)
```

Data Normalization transforming numeric columns to a standard scale. In machine learning, some feature values differ from others multiple times. The features with higher values will dominate the learning process.

Example:

```
import pandas as pd
```

```
# create data
```

```
df = pd.DataFrame({'Column 1':[200,-4,90,13.9,5,  
                               -90,20,300.7,30,-200,400],  
                  'Column 2':[20,30,23,45,19,38,  
                               25,45,34,37,12]})
```

```
# view data
```

```
display(df)
```

```
In [27]: import pandas as pd

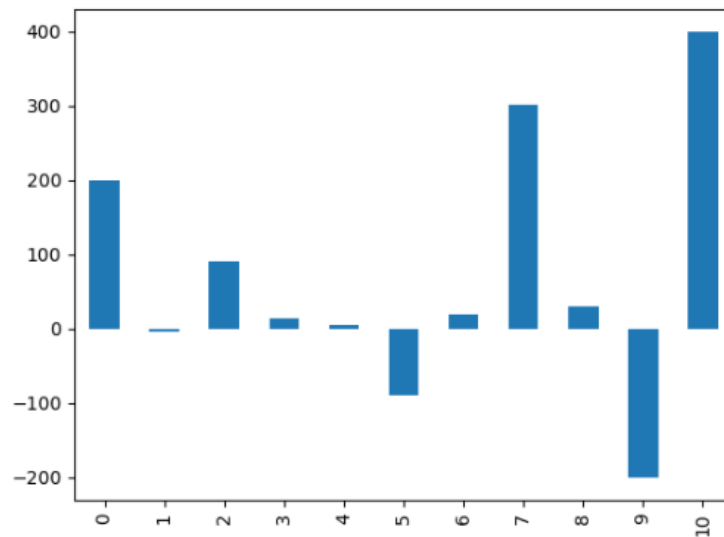
# create data
df = pd.DataFrame({'Column 1': [200, -4, 90, 13.9, 5,
                                -90, 20, 300.7, 30, -200, 400],
                   'Column 2': [20, 30, 23, 45, 19, 38,
                                25, 45, 34, 37, 12]})

# view data
display(df)
```

	Column 1	Column 2
0	200.0	20
1	-4.0	30
2	90.0	23
3	13.9	45
4	5.0	19
5	-90.0	38
6	20.0	25
7	300.7	45
8	30.0	34
9	-200.0	37
10	400.0	12

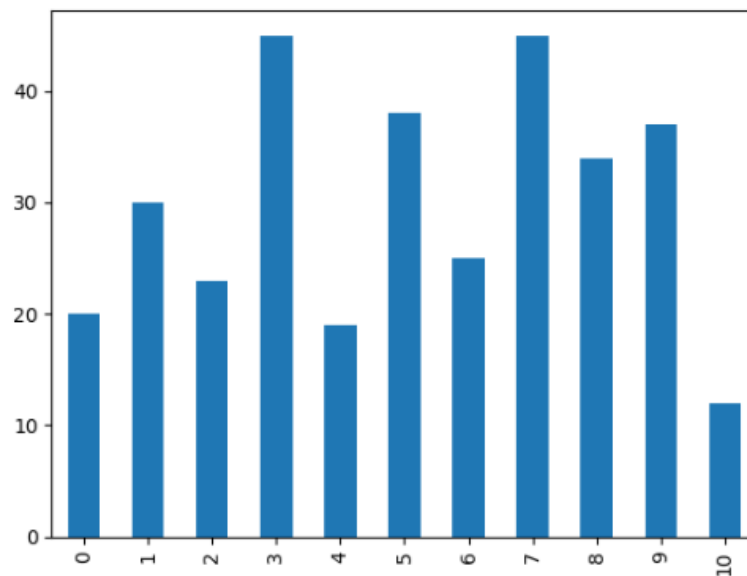
`df['Column 1'].plot(kind = 'bar')`

```
In [28]: df['Column 1'].plot(kind = 'bar')
Out[28]: <AxesSubplot>
```



```
In [29]: df['Column 2'].plot(kind = 'bar')
```

```
Out[29]: <AxesSubplot:>
```



```
In [ ]: |
```

It is clear that **column2** is more normalized than **column1**.

Normalize Using The maximum absolute scaling:

The **maximum absolute scaling** rescales each feature between **-1** and **1** by dividing every observation by its **maximum absolute** value. We can apply the maximum absolute scaling in Pandas using the **.max()** and **.abs()** methods.

```
# copy the data
```

```
df_max_scaled = df.copy()
```

```
# apply normalization techniques on Column 1
```

```
column = 'Column 1'
```

```
df_max_scaled[column] = df_max_scaled[column] / df_max_scaled[column].abs().max()
```

```
# view normalized data
```

```
display(df_max_scaled)
```



```
In [30]: # copy the data
df_max_scaled = df.copy()

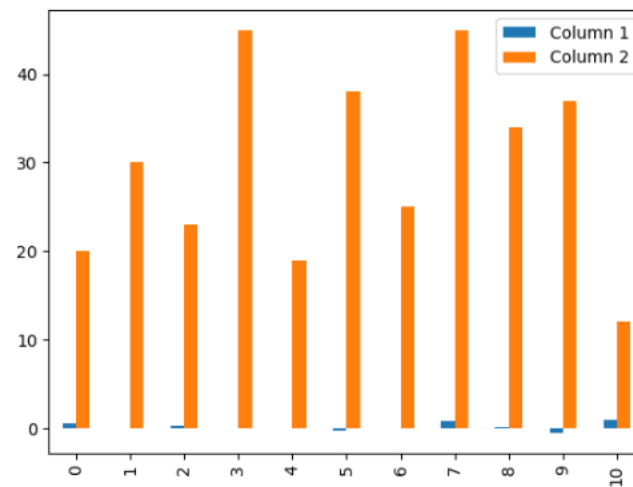
# apply normalization techniques on Column 1
column = 'Column 1'
df_max_scaled[column] = df_max_scaled[column] / df_max_scaled[column].abs().max()

# view normalized data
display(df_max_scaled)
```

	Column 1	Column 2
0	0.50000	20
1	-0.01000	30
2	0.22500	23
3	0.03475	45
4	0.01250	19
5	-0.22500	38
6	0.05000	25
7	0.75175	45
8	0.07500	34
9	-0.50000	37
10	1.00000	12

`df_max_scaled.plot(kind = 'bar')`

```
In [32]: df_max_scaled.plot(kind = 'bar')
Out[32]: <AxesSubplot:>
```



```
In [ ]:
```


Using The min-max feature scaling:

The min-max approach (often called normalization) rescales the feature to a hard and fast range of **[0,1]** by subtracting the minimum value of the feature then dividing by the range.

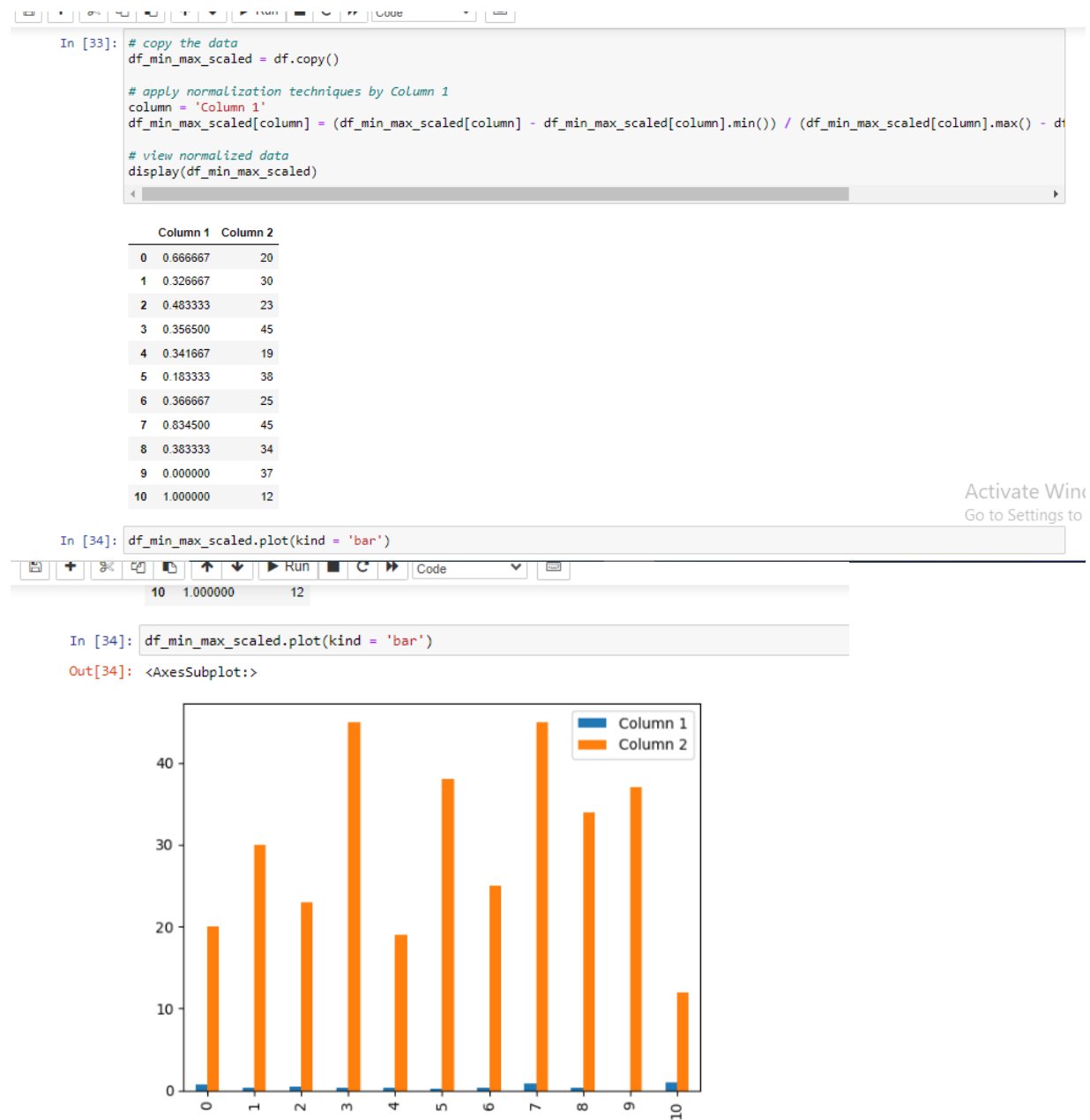
The diagram shows the min-max scaling process. It starts with a list of values $x = [1, 43, 65, 23, 4, 57, 87, 45, 45, 23]$. An arrow points down to the formula $x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$. To the right of the formula, the values $x_{min} = 1$ and $x_{max} = 87$ are specified. Another arrow points down to the resulting scaled values $x_{new} = [0, 0.48, 0.74, 0.25, 0.03, 0.65, 1, 0.51, 0.51, 0.25]$. The word "Activat" is partially visible at the bottom right of the diagram.

We can apply the min-max scaling in Pandas using the **.min()** and **.max()** methods.

```
# copy the data
df_min_max_scaled = df.copy()

# apply normalization techniques by Column 1
column = 'Column 1'
df_min_max_scaled[column] = (df_min_max_scaled[column] - df_min_max_scaled[column].min()) /
(df_min_max_scaled[column].max() - df_min_max_scaled[column].min())

# view normalized data
display(df_min_max_scaled)
```



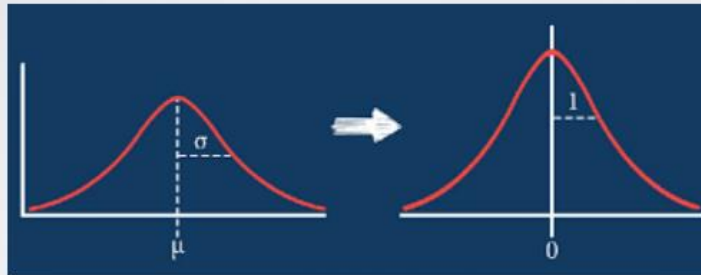
Using The z-score method:

The z-score method (often called standardization) transforms the info into distribution with a **mean of 0** and a **typical deviation of 1**. Each standardized value is computed by subtracting the mean of the corresponding feature then dividing by the quality deviation.

- **Data Standardisation:**

Standardisation typically means rescales data to have a mean of 0 and a standard deviation of 1 (unit variance).

$$x_{new} = \frac{x - \mu}{\sigma}$$



$x = [1, 43, 65, 23, 4, 57, 87, 45, 45, 23]$ $\mu = 39.3$ $x_{max} = 87$

↓
 $x_{new} = [-1.49, 0.14, 1.00, -0.63, -1.37, 0.69, 1.86, 0.22, 0.22, -0.63]$

```
# copy the data
```

```
df_z_scaled = df.copy()
```

```
# apply normalization technique to Column 1
```

```
column = 'Column 1'
```

```
df_z_scaled[column] = (df_z_scaled[column] - df_z_scaled[column].mean()) /  
df_z_scaled[column].std()
```

```
# view normalized data
```

```
display(df_z_scaled)
```

```
In [35]: # copy the data
df_z_scaled = df.copy()

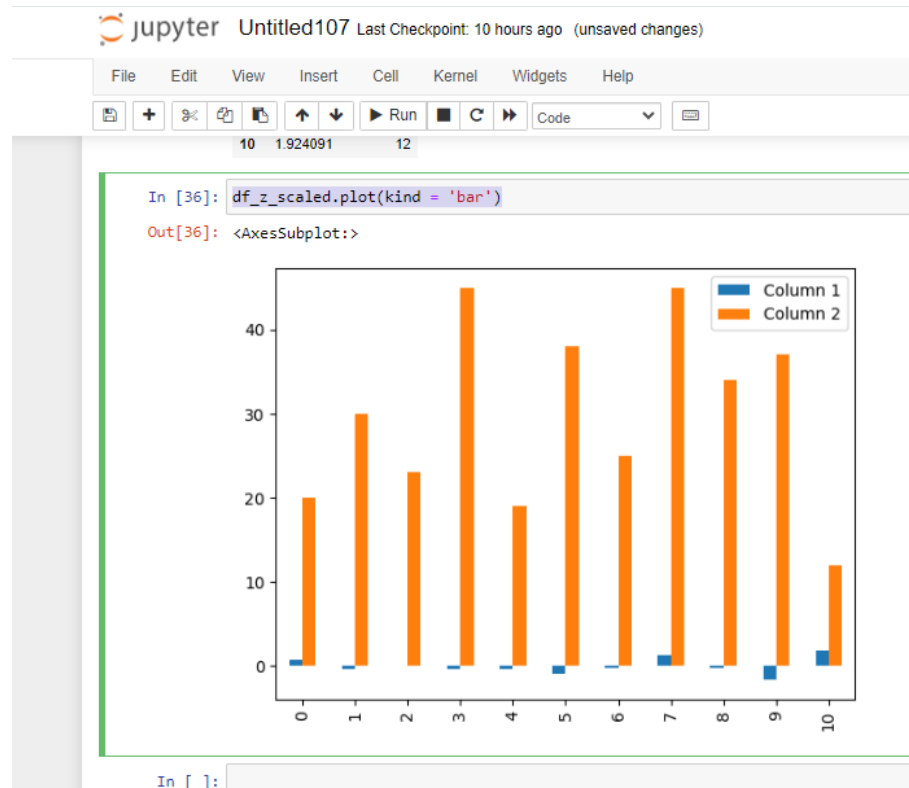
# apply normalization technique to Column 1
column = 'Column 1'
df_z_scaled[column] = (df_z_scaled[column] - df_z_scaled[column].mean()) / df_z_scaled[column].std()

# view normalized data
display(df_z_scaled)
```

	Column 1	Column 2
0	0.759387	20
1	-0.428611	30
2	0.118800	23
3	-0.324370	45
4	-0.376199	19
5	-0.929434	38
6	-0.288847	25
7	1.345815	45
8	-0.230611	34
9	-1.570021	37
10	1.924091	12

In []:

df_z_scaled.plot(kind = 'bar')



Example:

```
import pandas as pd

d = {"Name": ["Alisa", "Bobby", "Cat", "Madonna", "Rocky"],
     "Age": [1, 27, 25, 24, 31],
     "IQ": [100, 120, 95, 1300, 101]}

df = pd.DataFrame(d)

print(df.describe())
```

```
In [39]: import pandas as pd
d = {"Name": ["Alisa", "Bobby", "Cat", "Madonna", "Rocky"],
     "Age": [1, 27, 25, 24, 31],
     "IQ": [100, 120, 95, 1300, 101]}
df = pd.DataFrame(d)
print(df.describe())
```

	Age	IQ
count	5.000000	5.000000
mean	21.600000	343.200000
std	11.823705	534.952054
min	1.000000	95.000000
25%	24.000000	100.000000
50%	25.000000	101.000000
75%	27.000000	120.000000
max	31.000000	1300.000000

Example:

```
import pandas as pd

# list of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]

# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}
```

```
df = pd.DataFrame(dict)

print(df)

print()

print()

fm = df[(df.name == 'pankaj')]

print(fm)
```

```
In [53]: import pandas as pd
# list of name, degree, score
nme = ["aparna", "pankaj", "sudhir", "Geeku"]
deg = ["MBA", "BCA", "M.Tech", "MBA"]
scr = [90, 40, 80, 98]
|
# dictionary of lists
dict = {'name': nme, 'degree': deg, 'score': scr}

df = pd.DataFrame(dict)
print(df)
print()
print()
fm = df[(df.name == 'pankaj')]
print(fm)
```

	name	degree	score
0	aparna	MBA	90
1	pankaj	BCA	40
2	sudhir	M.Tech	80
3	Geeku	MBA	98

	name	degree	score
1	pankaj	BCA	40

Example:

```
airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")

airline_safety.head()
```

```
In [41]: airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")
airline_safety.head()
```

```
Out[41]:
```

	airline	avail_seat_km_per_week	incidents_85_99	fatal_accidents_85_99	fatalities_85_99	incidents_00_14	fatal_accidents_00_14	fatalities_00_14
0	Aer Lingus	320906734	2	0	0	0	0	0
1	Aeroflot*	1197672318	76	14	128	6	1	88
2	Aerolineas Argentinas	385803648	6	0	0	1	0	0
3	Aeromexico*	596871813	3	1	64	5	0	0
4	Air Canada	1865253802	2	0	0	2	0	0

```
In [ ]:
```

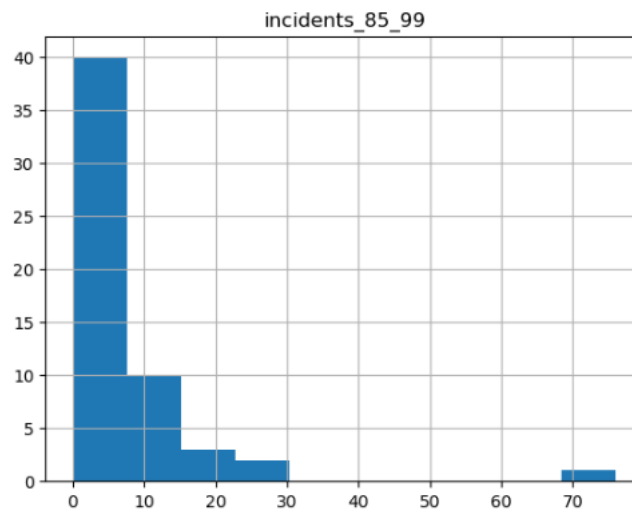
The data contains:

- airline: The new of the airline company.
- avail_seat: Passenger capacity. Available seat per km per week.
- incidents_85_99: Incidents between 1985 and 1999.
- fatal_accidents_85_99: Fatal accidents between 1985 and 1999.
- fatalities_85_99: Fatalities be between 1985 and 1999.
- incidents_00_14: Incidents between 2000 and 2014.
- fatal_accidents_00_14: Fatal accidents between 2000 and 2014.
- fatalities_00_14: Fatalities be between 2000 and 2014.

The data is stored in csv format and it appears to be structured (no missing data, no structural error).

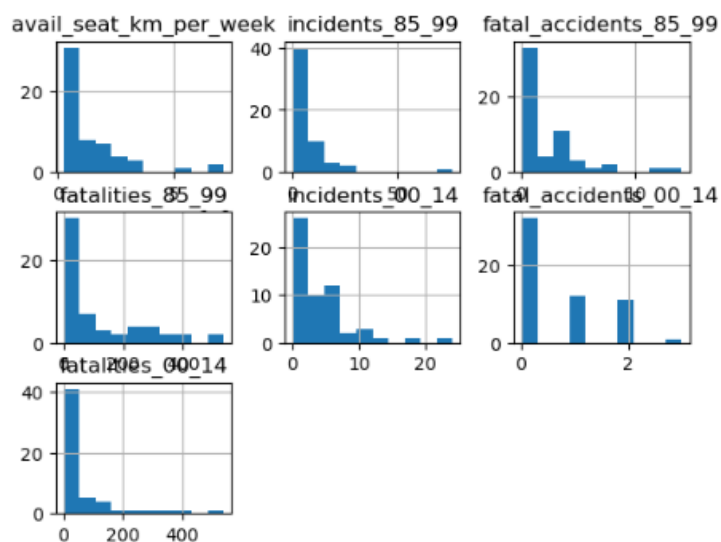
```
airline_safety.hist('incidents_85_99')
```

```
In [44]: airline_safety.hist('incidents_85_99')
Out[44]: array([[<AxesSubplot:title={'center':'incidents_85_99'}>]], dtype=object)
```



`airline_safety.hist()`

```
In [45]: airline_safety.hist()
Out[45]: array([[<AxesSubplot:title={'center':'avail_seat_km_per_week'}>,
<AxesSubplot:title={'center':'incidents_85_99'}>,
<AxesSubplot:title={'center':'fatal_accidents_85_99'}>],
[<AxesSubplot:title={'center':'fatalities_85_99'}>,
<AxesSubplot:title={'center':'incidents_00_14'}>,
<AxesSubplot:title={'center':'fatal_accidents_00_14'}>],
[<AxesSubplot:title={'center':'fatalities_00_14'}>],
<AxesSubplot:~>, <AxesSubplot:~>], dtype=object)
```




```
airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")
airline_safety.head()

# Filter out the airlines if incidents < 70
dfnan = df.mask(airline_safety["incidents_85_99"] < 70)

# Drop the irrelevant rows
df_filtered = dfnan.dropna()

# Print the results
print(df_filtered)
```

```
In [47]: airline_safety = pd.read_csv("H:/data science/AirlineSafetyDataset/airline-safety.csv")
airline_safety.head()
# Filter out the airlines if incidents < 70
dfnan = df.mask(airline_safety["incidents_85_99"] < 70)
# Drop the irrelevant rows
df_filtered = dfnan.dropna()
# Print the results
print(df_filtered)
```

	Name	Age	IQ
1	Bobby	27.0	120.0

```
In [ ]:
```